



Lab 02: Infrastructure as Code with Terraform

Build Infrastructure with Terraform | GSP750

terraform destroy

Overview

Built, changed, and destroyed infrastructure with Terraform. Created resource dependencies using implicit and explicit methods. Provisioned infrastructure using local-exec provisioners with Gemini Code Assist.

Objectives

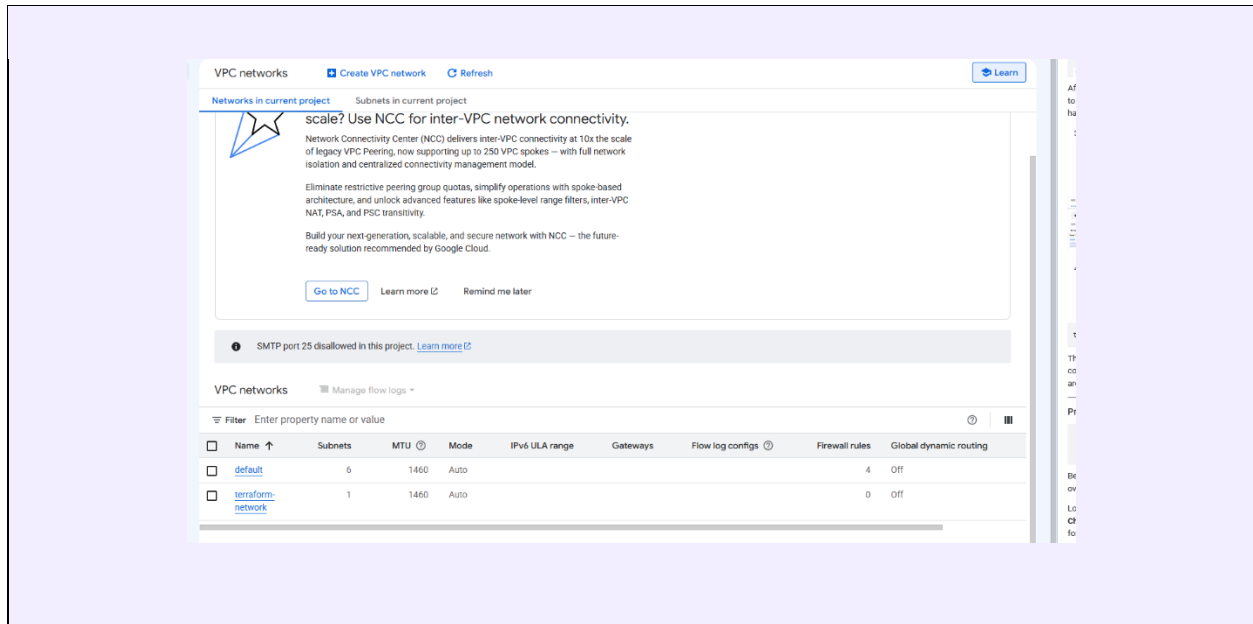
- Build and change infrastructure with Terraform
- Create resource dependencies
- Provision infrastructure with Terraform

Task 1 - Build Infrastructure

Created main.tf with provider and VPC network resource. Initialized and applied the configuration.

```
terraform {
  required_providers {
    google = { source = "hashicorp/google" version = "3.5.0" }
  }
}
provider "google" {
  project = "[PROJECT_ID]"
  region  = "[REGION]"
  zone    = "[ZONE]"
}
resource "google_compute_network" "vpc_network" {
  name = "terraform-network"
}
```

```
terraform init
terraform apply
```



Task 2 - Change Infrastructure

Added a VM instance resource, then used Gemini Code Assist to add tags and change the boot disk image.

```
resource "google_compute_instance" "vm_instance" {
  name         = "terraform-instance"
  machine_type = "e2-micro"
  tags         = ["web", "dev"]
  boot_disk {
    initialize_params { image = "cos-cloud/cos-stable" }
  }
  network_interface {
    network = google_compute_network.vpc_network.name
    access_config {}
  }
}
```

Destructive change: changing boot disk image requires destroying and recreating the VM. Terraform shows +/- prefix for destroy and recreate operations.

```

+ device_name      = (known after apply)
+ disk_encryption_key_sha256 = (known after apply)
+ kms_key_self_link = (known after apply)
+ mode             = "READ_WRITE"
+ source           = (known after apply)

+ initialize_params {
+   image = "debian-cloud/debian-12"
+   labels = (known after apply)
+   size   = (known after apply)
+   type   = (known after apply)
+ }

+ network_interface {
+   name            = (known after apply)
+   network         = "terraform-network"
+   network_ip      = (known after apply)
+   subnetwork      = (known after apply)
+   subnetwork_project = (known after apply)
+ }

+ access_config {
+   nat_ip        = (known after apply)
+   network_tier  = (known after apply)
+ }
}

Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

google_compute_instance.vm_instance: Creating...
google_compute_instance.vm_instance: Still creating... [10s elapsed]
google_compute_instance.vm_instance: Creation complete after 16s [id=projects/qwiklabs-gcp-04-034d7e042e08/zones/us-east1-c/instances/terraform-instance]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
student_01_e9e900c99425@cloudshell:~ (qwiklabs-gcp-04-034d7e042e08) $

```

Task 3 - Create Resource Dependencies

Added a static IP address and connected it to the VM using implicit dependency via interpolation.

```

resource "google_compute_address" "vm_static_ip" {
  name = "terraform-static-ip"
}

network_interface {
  network = google_compute_network.vpc_network.self_link
  access_config { nat_ip = google_compute_address.vm_static_ip.address }
}

```

Also added explicit dependency using `depends_on` for a Cloud Storage bucket and second VM instance.

```

resource "google_compute_instance" "another_instance" {
  depends_on = [google_storage_bucket.example_bucket]
  ...
}

```

```

terraform plan -out static_ip
terraform apply "static_ip"

```

```

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.
student_01_e9e900c99425@cloudshell:~ (qwiklabs-gcp-04-034d7e042e08)$ resource "google_compute_address" "vm_static_ip" {
  name = "terraform-static-ip"
}
-bash: resource: command not found
-bash: name: command not found
-bash: syntax error near unexpected token `}'
student_01_e9e900c99425@cloudshell:~ (qwiklabs-gcp-04-034d7e042e08)$ terraform plan
google_compute_network.vpc network: Refreshing state... [id=projects/qwiklabs-gcp-04-034d7e042e08/global/networks/terraform-network]
google_compute_instance.vm_instance: Refreshing state... [id=projects/qwiklabs-gcp-04-034d7e042e08/zones/us-east1-c/instances/terraform-instance]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# google_compute_address.vm_static_ip will be created
+ resource "google_compute_address" "vm_static_ip" {
  + address            = (known after apply)
  + address_type       = "EXTERNAL"
  + creation_timestamp = (known after apply)
  + id                 = (known after apply)
  + name               = "terraform-static-ip"
  + network_tier        = (known after apply)
  + project             = (known after apply)
  + purpose             = (known after apply)
  + region             = (known after apply)
  + self_link           = (known after apply)
  + subnetwork          = (known after apply)
  + users               = (known after apply)
}

Plan: 1 to add, 0 to change, 0 to destroy.

```

Task 4 - Provision Infrastructure

Added a local-exec provisioner to write the VM name and IP address to a file on apply.

```

provisioner "local-exec" {
  command = "echo ${google_compute_instance.vm_instance.name}:
${google_compute_instance.vm_instance.network_interface[0].access_config[0].nat_ip}
>> ip_address.txt"
}
terraform taint google_compute_instance.vm_instance
terraform apply

```

```

Terraform has compared your real infrastructure against your configuration and found no differences, so no changes are needed.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.
student_01_e9e900c99425@cloudshell:~ (qwiklabs-gcp-04-034d7e042e08)$ cat ip_address.txt
terraform-instance: 34.75.32.142
student_01_e9e900c99425@cloudshell:~ (qwiklabs-gcp-04-034d7e042e08)$ 

```

What I Learned

- Terraform tracks changes and only modifies what needs to change
- The ~ prefix means in-place update, -/+ means destroy and recreate
- Implicit dependencies are created when one resource references another's attribute
- Explicit dependencies use depends_on when the relationship is not visible to Terraform
- Provisioners run commands locally or on the remote resource after creation
- terraform taint forces a resource to be destroyed and recreated on next apply

Resources

